Pure Pursuit

Contents

1. Overview . . . 2
2. Chasing a point on the path . . . 4
3. The arc through the look-ahead point . . . 5
   - 3.1 Why an arc, and which one . . . 5
   - 3.2 The radius from one right triangle . . . 6
   - 3.3 From curvature to steering angle . . . 8
4. Pure pursuit as a loop . . . 9
   - 4.1 Find the point, steer, drive, repeat . . . 9
   - 4.2 Why it does not weave: the look-ahead point sees the heading . . . 10
5. Choosing the look-ahead distance . . . 14
   - 5.1 Short or long, on the chapter's car . . . 14
   - 5.2 A real steering motor: the short look-ahead weaves . . . 16
6. A look-ahead that grows with speed . . . 18
7. Regulated pure pursuit: slow down where the path is tight . . . 20
8. Where pure pursuit runs: Nav2 and Autoware . . . 23
9. Summary . . . 24
10. Sources . . . 25
11. Key terms . . . 26

Where this fits:

- **Builds on:** Kinematic bicycle model; Lateral and heading error.
- **Leads to:** Comparing and choosing trackers.
- **Compare with:** Stanley controller.

1. Overview

Key point: RO-070's line rule turns the car in proportion to its two errors, so to turn with the 10 m arc it must stay up to 0.39 m outside it. Pure pursuit steers along the arc to a point on the path a fixed distance ahead, so it sees a bend coming and turns with it (0.03 m off by the middle of the arc), and its one setting, the look-ahead distance, trades weaving against cutting corners.

So far the chapter can measure where the car is against its path and steer on that. Path coordinates (G-3132) give, for any pose, the nearest path point (G-3134), the **lateral error** (G-3135), the signed distance from the path (positive to the left), and the **heading error** (G-3136), the heading minus the path's direction there. A tracker needs both errors. RO-070's demonstration driver, the line rule applied at the nearest point, turns the car towards the path at 0.5 rad/s per metre of lateral error and 2 rad/s per radian of heading error, and from 1.0 m off it is back within 0.1 m after 7.70 m (measuring a run).

What still breaks is the arc. The line rule turns only in proportion to its errors, so to turn with the 10 m arc it needs an error: it runs up to 0.39 m outside the arc, the whole way round (measuring a run). A driver does not wait to be off the road before turning; a driver looks down the road and sees the bend coming. The car of the Hybrid A* planner already followed its route that way, steering towards the route point 2 m ahead. This Note names that rule, pure pursuit, and explains why it works.

The car and its test course are the chapter's:

- a car with 2.5 m between its axles, the **wheelbase** (G-3127), that steers its front wheels by at most 26.6°, so its tightest circle has a radius of 5 m;
- the kinematic bicycle model (G-3128): the two front wheels drawn as one steered wheel and the two rear wheels as one, with no tyre slip, which is close enough at these speeds (at most 2.5 m/s^2 sideways, on the arc at 5 m/s);
- the course: 10 m straight along x from (0, 0), a 90° left arc of radius 10 m, then 10 m straight;
- the start: rear axle at (0, 1.0), 1.0 m left of the line, heading along it;
- speed 2 m/s, and a new steering command 20 times a second, Nav2's default rate (a sample rate (G-3106) of 20 Hz).

Every run in this Note is our own Python model of the rules on this car (the notebook `RO-071-pure-pursuit.ipynb`), not the code of a real robot stack; Section 8 reads the real code.

Figure 1 drives the course with both rules. Watch the two cars on the arc: the red car, the line rule, runs outside it all the way round; the blue car chases a point 4 m ahead on the

path.

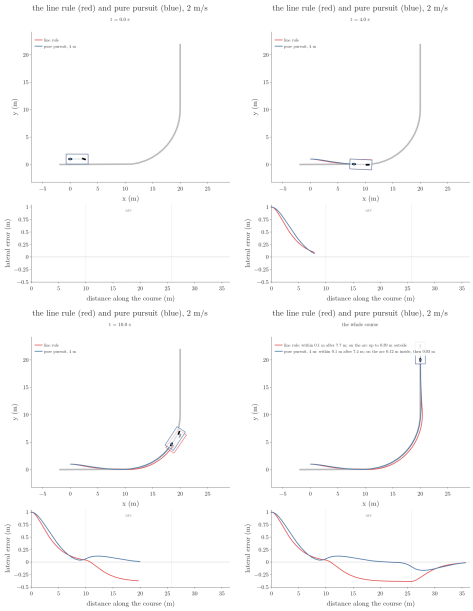


Figure 1: RO-070's line rule (red; turn rate $-0.5\,\mathrm{e} - 2\,\theta\mathrm{e}$ from the lateral error e and the heading error θe) and pure pursuit with a 4 m look-ahead (blue), both from 1.0 m left of the course at 2 m/s; one frame per 0.5 s. Top: the paths of the rear axles. Bottom: the lateral error along the course (+ = left), with the arc between the dotted lines. Last frame: the line rule is within 0.1 m after 7.7 m but runs up to 0.39 m outside the arc; pure pursuit is within 0.1 m after 7.4 m, runs 0.12 m inside at the arc's entry and 0.03 m off by its middle

The blue car comes back from 1.0 m off a little sooner than the red one, within 0.1 m after 7.4 m against 7.70 m. On the arc it cuts 0.12 m inside at the entry, and by the middle of the arc it is 0.03 m off, where the red car is 0.36 m outside (notebook).

This Note builds pure pursuit in the order a driver would explain it:

- the point to chase, one look-ahead distance ahead on the path (Section 2);
- the arc that reaches it, and the steering angle that drives that arc (Section 3);
- pure pursuit as a loop at 20 Hz, why it turns with the arc and why it does not weave (Section 4);
- choosing the look-ahead distance: weaving against cutting corners (Section 5);
- a look-ahead that grows with speed (Section 6);
- regulated pure pursuit: slowing down on tight curves and near obstacles (Section 7);
- where it runs: Nav2 and Autoware (Section 8).

2. Chasing a point on the path

> **Key point:** The line rule steers from where the car is; pure pursuit steers towards where the car should be a little later: the point on the path a fixed distance ahead of the rear axle, the look-ahead point. At the start, with a 4 m look-ahead, that point is (3.873, 0).

A driver does not steer by the paint under the car. A driver looks some way down the road and heads for that spot, and the spot moves on as the car moves (Coulter §1.2). Pure pursuit does the same: the car chases a point on the path some distance ahead, like a donkey chasing a carrot on a stick, and the name comes from that chase (Coulter §1.2; UW CSE 478 L11).

Two things are assumed known (Coulter §3.1):

- the path, as a list of points with their positions, such as our course sampled every few millimetres;
- the car's pose, from its localisation.

The point to chase is found with a circle. Draw a circle around the rear axle; where it meets the path ahead, the car aims. The circle's radius is the **look-ahead distance** (G-3151), written l_d, and the point is the **look-ahead point** (G-3152), also called the goal point or carrot (Coulter §2.0; Nav2 RPP README). The circle can meet the path more than once, so the search starts at the path point nearest the car and walks forward along the path until the distance from the rear axle reaches l_d (Coulter §3.3).

Figure 2 shows the start. The rear axle is 1.0 m above the line $y = 0$, and the look-ahead distance is 4 m.

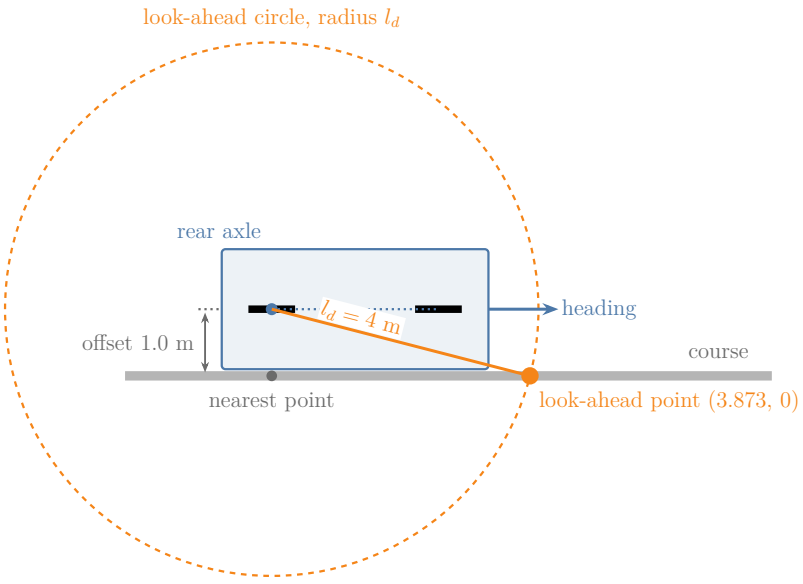


Figure 2: The idea of pure pursuit at the start: the car's rear axle (blue dot) is 1.0 m left of the course, heading along it. The dashed circle of radius 4 m around the rear axle meets the course ahead at the look-ahead point (orange), 3.873 m along; the car will chase that point

The look-ahead point lies on the line, 4 m from the rear axle, which sits 1 m above the line. With:

- l_d, the look-ahead distance, 4 m;
- x, how far along the line the point is, in m.

By Pythagoras:

$$x^2 + 1^2 = 4^2$$

$$x^2 = 15$$

$$x = 3.873 \text{ m}$$

If the car is farther from the path than l_d, the circle misses the path and the rule has no point to chase (Paden §V.A.1); implementations then aim at the nearest path point, as ours does.

The car now knows where to head. But a car cannot turn on the spot to face the point; it can only drive forward along arcs. The next question is which arc reaches the point.

3. The arc through the look-ahead point

> **Key point:** With the steering held still, the car drives a circle, so in one command it can only drive an arc. The arc that leaves the car along its heading and passes through the look-ahead point has a curvature of twice the point's sideways distance y over the look-ahead distance l_d squared: -0.125 per m at the start, a circle of 8 m to the right, which the car drives with the steering at $-17.4°$.

3.1 Why an arc, and which one

The look-ahead point is 3.873 m ahead of the rear axle and 1 m to its right. In the kinematic bicycle model (G-3128), a car whose steering is held at one angle drives round one circle (Snider §2.1). So between two steering commands the car drives an arc, and pure pursuit picks the arc that ends at the look-ahead point.

Many arcs join the rear axle and the point. Only one of them can be driven: at this instant the rear axle moves straight along the heading, because the rear wheel cannot slide sideways. So the arc must leave the rear axle along the heading. The centre of an arc that starts along the heading lies square to it, on the line through the rear axle. The rear-axle line is where the turning car's instantaneous centre of curvature (G-2529), the point the whole car turns about, has to be, for the same reason as for the differential drive (Snider §2.2; Coulter §2.0).

We measure the look-ahead point from the car itself: from the rear axle, x forward and y to the left, the car's own frame. A point at world position (x_w, y_w), seen from a car at (x_r, y_r) with heading θ:

- x is the point's distance ahead of the rear axle, in m;
- y is its distance to the left of the heading line, in m (negative: to the right).

$$x = \cos\theta\,(x_w - x_r) + \sin\theta\,(y_w - y_r)$$

$$y = -\sin\theta\,(x_w - x_r) + \cos\theta\,(y_w - y_r)$$

At $\theta = 0$, the cosine is 1 and the sine is 0, so x and y are just the gaps along the world axes, as they should be. At the start the point is at $x = 3.873$ m and $y = -1$ m.

3.2 The radius from one right triangle

Figure 3 draws the arc in the car's frame. Its centre sits on the rear-axle line at a signed distance R from the rear axle, positive to the left, so the centre is at $(0, R)$. The look-ahead point is on the arc, so it is $|R|$ from the centre: the green right triangle has the legs x and $y - R$ and the long side R.

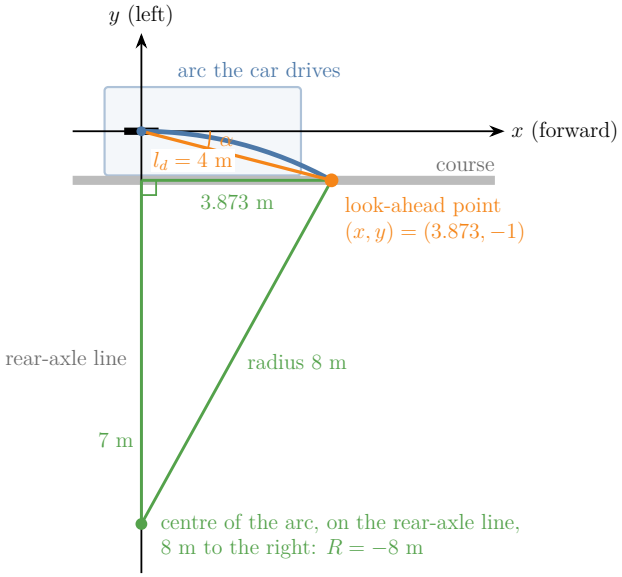


Figure 3: The arc through the look-ahead point in the car's frame at the start: x forward, y left, origin at the rear axle. The look-ahead point (orange) is at (3.873, −1), 4 m from the rear axle (the chord, orange). The arc the car drives (blue) leaves the rear axle along the heading, so its centre (green) lies on the rear-axle line, 8 m to the right. The green right triangle has legs 7 m and 3.873 m and the long side 8 m, the radius

With:

- x, the look-ahead point's distance ahead, 3.873 m, so $x^2 = 15$ m^2;
- y, its sideways distance, positive to the left, −1 m;

- l_d, the look-ahead distance, 4 m, with $x^2 + y^2 = l_d^2$;
- R, the signed radius of the arc, positive when the centre is to the left, in m;
- κ, the arc's curvature, one over its radius, in 1/m: the **path curvature** (G-3129) of the arc the car will drive.

The point's distance from the centre equals the radius; our numbers first, then the same steps in symbols:

$$15 + (-1 - R)^2 = R^2$$

$$15 + 1 + 2R + R^2 = R^2$$

$$16 + 2R = 0$$

$$R = -8 \text{ m}$$

$$\kappa = 1/(-8) = -0.125 \text{ per m}$$

$$x^2 + (y - R)^2 = R^2$$

$$x^2 + y^2 - 2yR + R^2 = R^2$$

$$l_d^2 - 2yR = 0$$

$$R = \frac{l_d^2}{2y}$$

$$\kappa = \frac{2y}{l_d^2}$$

The arc is a circle of radius 8 m with its centre to the right (Coulter §2.0, eqs. 2.1–2.2). Nav2's `calculateCurvature` computes the same thing, since $x^2 + y^2$ is l_d^2:

$$\kappa = \frac{2y}{x^2 + y^2}$$

The same curvature can be written with the angle α between the heading and the line to the look-ahead point (Figure 3). With:

- α, the angle from the heading to the look-ahead point, positive to the left: $-14.5°$ at the start.

The point's sideways distance is the look-ahead distance times the sine of that angle:

$$\sin\alpha = -1/4 = -0.25$$

$$\kappa = 2 \times (-0.25)/4 = -0.125$$

$$y = l_d \sin\alpha$$

$$\kappa = \frac{2\sin\alpha}{l_d}$$

The angle form is the textbook one (Snider §2.2, eq. 2; Paden §V.A.1, eq. V.2). Two checks at easy values:

- **The point straight ahead,** $\alpha = 0$: the curvature is 0, and the car drives straight, as it should.
- **The point square to the left,** $\alpha = 90°$: the curvature is 2 over l_d, a circle whose radius is half of l_d, so its diameter is the line from the rear axle to the point.

3.3 From curvature to steering angle

The car does not take a curvature; it takes a steering angle. In the bicycle model the rear axle circles with radius R when the front wheel is turned by the angle whose tangent is the wheelbase over R (Snider §2.1, eq. 1), so the tangent of the **steering angle** (G-3126) is the wheelbase times the curvature. With:

- L, the wheelbase, 2.5 m;
- κ, the curvature of the arc, -0.125 per m;
- δ, the steering angle of the front wheel, positive to the left, at most $26.6°$ either way.

$$\tan\delta = 2.5 \times (-0.125)$$

$$= -0.3125$$

$$\delta = -17.4°$$

$$\tan\delta = L\,\kappa$$

$$\delta = \tan^{-1}(L\,\kappa)$$

Put together, the steering from the angle to the look-ahead point is (Snider §2.2):

$$\delta = \tan^{-1}\left(\frac{2L\sin\alpha}{l_d}\right)$$

The car steers 17.4° to the right, within its 26.6°. The steering answers one moment. But after one tick, 0.05 s later, the car has moved 0.1 m and the look-ahead point is somewhere else, so the arc has to be found again.

4. Pure pursuit as a loop

> **Key point:** Pure pursuit repeats the same steps at every tick: find the look-ahead point, compute the arc, set the steering, drive. From 1.0 m off, the 4 m look-ahead brings the car within 0.1 m after 7.4 m without crossing the line, because the look-ahead point's sideways distance contains the heading error as well as the lateral error: near a straight path, a spring and a damper together, with a damping ratio of 0.707 for any look-ahead distance.

4.1 Find the point, steer, drive, repeat

The arc of Section 3 is right only for the instant it was computed. So the rule runs as a loop, 20 times a second (Coulter §3.3):

1. Read the car's pose.
2. Find the path point nearest the rear axle.
3. Walk forward along the path to the look-ahead point, the path point one look-ahead distance from the rear axle.
4. Turn the point into the car's frame (Section 3.1).
5. Compute the curvature and set the steering (Sections 3.2 and 3.3); a steering angle beyond the car's 26.6° is clipped to 26.6°, and our runs drive with the clipped value.
6. Hold that steering until the next tick, 0.05 s later (a zero-order hold (G-3109)), and go back to step 1.

The loop is **pure pursuit** (G-3150) (Coulter 1992; Paden §V.A.1). As the car turns towards the look-ahead point, the point moves on along the path, so the steering eases off and the car slides onto the path.

Figure 4 runs the loop over the whole course, drawing every tenth tick. Watch the green arc, the arc the car is told to drive. At the start it bends right with a radius of 8 m. By 3.0 s the car is still 0.23 m left of the line, but the arc already bends left, with a radius of 37.3 m: the car straightens before it reaches the line. On the arc, at 10 s, the green arc has a radius of 9.7 m, close to the course's 10 m.

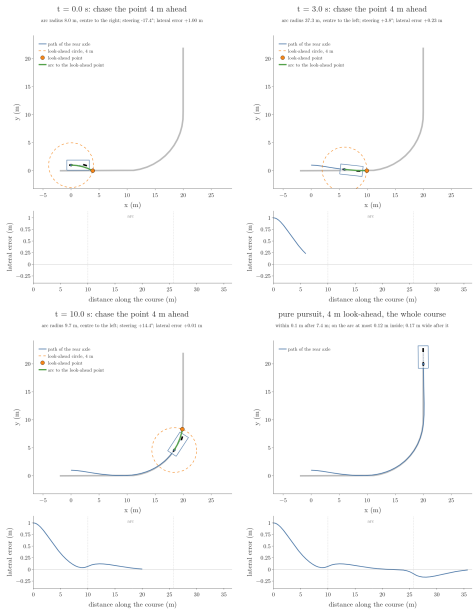


Figure 4: Pure pursuit with a 4 m look-ahead at 2 m/s, one frame every 0.5 s (every tenth tick of the 20 Hz loop). Top: the dashed circle of radius 4 m around the rear axle, the look-ahead point (orange) where it meets the course ahead, and the arc to that point (green), which the car drives until the next tick. Bottom: the lateral error along the course (+ = left). Frame t = 3.0 s: still 0.23 m left, the car already steers left (arc radius 37.3 m). Last frame: within 0.1 m after 7.4 m; on the arc at most 0.12 m inside; 0.17 m wide after it

The numbers of the run (notebook):

- **The return:** within 0.1 m of the line after 7.4 m, without crossing it before the arc;
- **on the arc:** at most 0.12 m inside the turn, 0.03 m by its middle;
- **after the arc:** 0.17 m wide of the last straight, then back;
- **steering:** at most 17.4°, at the start.

The run answers the opening problem on the arc: by its middle the car is 0.03 m off, where the line rule stayed 0.36 m outside. Once the look-ahead point is on the arc, the arc that leaves the rear axle along the heading and reaches the point is the course's own arc: on a circle, any look-ahead distance shorter than its diameter gives exactly the circle's curvature (Coulter §4.2). From 1 m into our arc, look-aheads of 2, 4 and 8 m all command 0.100 per m (notebook). So pure pursuit turns with the arc without needing an error; its errors come where the curvature changes, at the arc's two ends.

Like the line rule, pure pursuit also comes back from 1.0 m off without swinging past the line, though it has one setting instead of the line rule's two gains. Why it does not swing is the next question.

4.2 Why it does not weave: the look-ahead point sees the heading

At 3.0 s in Figure 4 the car is still 0.23 m left of the line, yet pure pursuit already steers left, braking its return. A tracker needs both errors, and pure pursuit measures neither: it gets

both from the look-ahead point. Figure 5 takes a car 0.5 m left of the line and pointing 0.1 rad (5.7°) further left. Measured from the car's heading line, the look-ahead point 4 m ahead is about 0.9 m to the right: 0.5 m because the car is off the line, and 0.4 m more because the car points away from it.

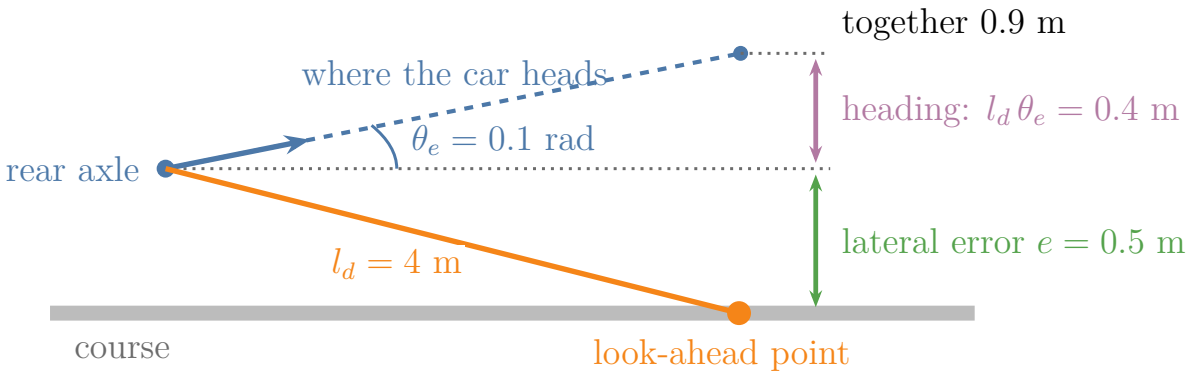


Figure 5: Why the look-ahead point sees the heading. The rear axle is 0.5 m left of the course (lateral error e) and heads 0.1 rad to the left of it (heading error). 4 m along its heading the car would be 0.9 m from the course, so the look-ahead point lies about 0.9 m to the right of the heading line: 0.5 m from the lateral error (green) and 0.4 m from the heading (purple). Sideways distances are drawn twice as large as forward ones

Near a straight path, for small angles, with:

- e, the rear axle's lateral error, positive to the left: 0.5 m;
- θ_e, the heading error, positive to the left: 0.1 rad;
- l_d, the look-ahead distance, 4 m;
- y, the look-ahead point's sideways distance in the car's frame, in m;
- κ, the curvature pure pursuit commands, in 1/m.

$$y \approx -(0.5 + 4 \times 0.1)$$

$$= -0.9 \text{ m}$$

$$\kappa = 2 \times (-0.9)/16$$

$$= -0.1125 \text{ per m}$$

$$y \approx -(e + l_d\,\theta_e)$$

$$\kappa = -\frac{2}{l_d^2}\, e - \frac{2}{l_d}\, \theta_e$$

The exact values are -0.894 m and -0.112 per m (notebook). So pure pursuit's curvature has two parts: one in proportion to the lateral error and one in proportion to the heading error, like the line rule's two terms. Coulter (§2.0) and Snider (§2.2, eq. 4) call pure pursuit a proportional controller on the sideways distance of a point ahead, with the gain 2 over l_d squared; the heading part comes from that point being ahead.

The heading error is how fast the lateral error grows as the car drives, so the second part acts like derivative action (G-2960), a damper. To see how much it damps, we follow the lateral error along the distance driven instead of time. With:

- s, the distance driven along the path, in m;
- e', how fast the lateral error changes per metre driven, which for small angles is the heading error θ_e;
- e'', how fast e' changes per metre, which on a straight path is the curvature the car drives, κ.

For the 4 m look-ahead the two factors are:

$$2/l_d^2 = 2/16 = 0.125$$

$$2/l_d = 2/4 = 0.5$$

$$e'' = -0.125\, e - 0.5\, e'$$

$$e'' + 0.5\, e' + 0.125\, e = 0$$

$$e'' + \frac{2}{l_d}\, e' + \frac{2}{l_d^2}\, e = 0$$

The last line is the equation of a block on a spring and a damper, with metres driven in place of seconds. Matched against the standard form of natural frequency and damping ratio, with:

- ω_n, the **natural frequency** (G-2991), here in radians per metre driven;
- ζ, the **damping ratio** (G-2992), with no unit.

$$\omega_n^2 = 0.125$$

$$\omega_n = 0.3536 \text{ per m}$$

$$2\zeta\omega_n = 0.5$$

$$\zeta = 0.5/(2 \times 0.3536)$$

$$= 0.707$$

$$\omega_n = \sqrt{2}/l_d$$

$$\zeta = \frac{2/l_d}{2\sqrt{2}/l_d} = \frac{1}{\sqrt{2}}$$

The look-ahead distance cancels: every look-ahead gives the damping ratio 0.707. By the overshoot formula, a damping ratio of 0.707 swings past by 4.3 percent, so a car starting 1.0 m off would cross the line by about 0.043 m. With the 4 m look-ahead the swing past would peak at the peak time counted in metres, π times the look-ahead, 12.6 m down the course, after the arc has begun, which is why Figure 4 shows no crossing on the first straight; the 2 m look-ahead of Section 5 crosses by 0.042 m.

The line rule of Figure 1 has the same two parts with gains chosen one by one: it turns at -0.5 rad/s per metre of lateral error and -2 rad/s per radian of heading error, and at 2 m/s the turn per metre driven is the turn rate over the speed:

$$e'' = (-0.5\,e - 2\,e')/2$$

$$e'' + 1.0\,e' + 0.25\,e = 0$$

$$\omega_n = 0.5 \text{ per m}$$

$$\zeta = 1.0/(2 \times 0.5) = 1.0$$

a damping ratio of 1, which just stops the swing (critically damped (G-2995)). Pure pursuit takes both gains from one number, the look-ahead distance, always in the same ratio.

Pure pursuit's damping is fixed, but its look-ahead distance still sets how far the car drives before it settles, and what happens where the path bends. The next section varies it.

5. Choosing the look-ahead distance

> **Key point:** On the chapter's car a short look-ahead brings the car back sooner but asks for sharp steering, and a long one cuts corners: 8 m cuts 0.85 m inside the arc. Once we give the car a real steering motor's lag of 0.4 s, the delay that makes real cars weave, at 5 m/s a 2 m look-ahead weaves 1.25 m off the line, while 8 m still cuts 0.56 m inside: the look-ahead trades weaving against cutting corners.

5.1 Short or long, on the chapter's car

The 4 m look-ahead distance, the radius of the circle that picks the chased point, brought the car back in 7.4 m but cut 0.12 m inside the arc. Figure 6 runs the same course with look-aheads of 2, 4 and 8 m. Watch the purple 8 m car at the arc's entry: it is still 0.63 m off when it starts to turn, and it turns early, inside the arc.

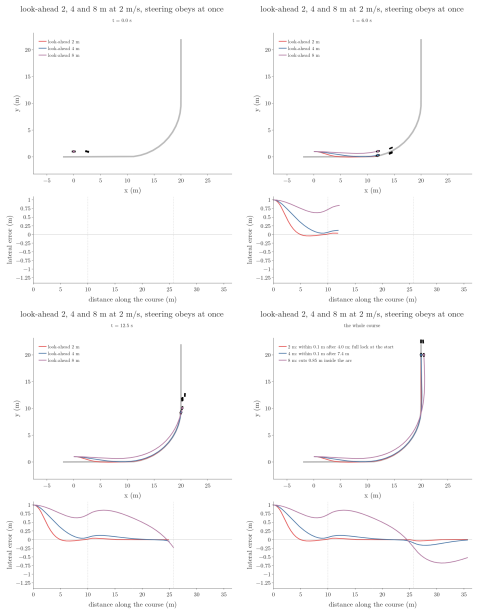


Figure 6: The look-ahead distance swept at 2 m/s on the chapter's car, whose wheels turn the instant they are told: 2 m (red), 4 m (blue), 8 m (purple), all from 1.0 m left of the course; one frame per 0.5 s. Top: the paths of the rear axles. Bottom: the lateral errors along the course (+ = left). Last frame: 2 m is within 0.1 m after 4.0 m but steers at full lock at the start; 4 m after 7.4 m; 8 m cuts 0.85 m inside the arc

The three runs (notebook):

Look-ahead	Within 0.1 m after	Past the line, first straight	Largest on the arc	After the arc	Largest steering
2 m	4.0 m	0.04 m	0.04 m	0.04 m	26.6° (full lock)
4 m	7.4 m	none	0.12 m inside	0.17 m wide	17.4°

Look-ahead	Within 0.1 m after	Past the line, first straight	Largest on the arc	After the arc	Largest steering
8 m	23.9 m	none	0.85 m inside	0.68 m wide	14.2°

What each one shows:

- **A short look-ahead returns sooner.** The natural frequency ω_n of Section 4.2 is $\sqrt{2}$ divided by l_d, so it doubles when l_d halves, and the car settles in about half the distance.
- **A short look-ahead asks for sharp steering.** From 1.0 m off, the 2 m look-ahead puts the look-ahead point 1 m to the right of the heading line at only 2 m, which asks for more than the car can steer (worked below), so the wheels sit at full lock, 26.6°.
- **A long look-ahead cuts corners.** At the arc's entry the look-ahead point is already on the arc while the car is still on the straight. The one arc through it starts to turn early, so the car runs inside (Coulter §4.1; Snider §2.2.1). At the exit the point is already on the last straight while the car is still on the arc, so the car straightens early and swings wide.
- **Inside a long arc the error dies away,** whatever the look-ahead, because the arc through the look-ahead point is the course's own arc (Section 4.1). The long look-ahead's errors come from where the curvature changes.

The sharp steering of the 2 m look-ahead at the start, with the point's sideways distance y of -1 m, the look-ahead distance l_d of 2 m and the wheelbase L of 2.5 m (Section 3):

$$\kappa = 2 \times (-1)/2^2 = -0.5 \text{ per m}$$

$$\tan\delta = 2.5 \times (-0.5) = -1.25$$

$$\delta = -51.3°$$

which the car cannot steer, so the command is clipped to $-26.6°$, full lock to the right, and the run drives with that.

Speed changes nothing here. At 5 m/s the three cars drive almost the same paths: within 0.1 m after 3.9, 7.2 and 23.8 m (notebook). Pure pursuit computes a curvature, not a turn rate, and the bicycle model turns a curvature into the same circle at any speed; Macenski et al. (§3) put it as "Pure Pursuit itself is independent of velocity". The small differences come from the ticks: 0.1 m driven per tick at 2 m/s, 0.25 m at 5 m/s.

On this car the shortest look-ahead looks best, apart from its sharp steering. Real cars weave with a short look-ahead (Coulter §4.1; Snider §2.2.1), because their wheels do not turn at once: pure pursuit assumes the car takes each curvature the moment it is asked, and a real steering motor answers with a lag (Coulter §4.3).

5.2 A real steering motor: the short look-ahead weaves

The chapter's car turns its wheels the instant it is told; a real steering motor needs time. The team of Stanford's car, Stanley, modelled its steering as a first-order system (G-2987) with a time constant (G-2988) of 0.4 s, a cautious value from their experiments (Hoffmann et al. 2007, §II.B, eq. 4). From here to Section 7 we change the chapter's car: it gets the same steering lag. Without the lag, nothing in the simulation would show why the look-ahead must change with speed. With:

- δ_c, the steering angle commanded, in degrees;
- δ, the angle the wheels are really at, in degrees;
- $\dot{\delta}$, how fast the wheels turn, in degrees per second;
- τ, the time constant, 0.4 s.

$$\dot{\delta} = (\delta_c - \delta)/\tau$$

The wheels close 63 percent of the gap in each 0.4 s. At the start, with the 4 m look-ahead, the command jumps to $-17.4°$:

$$0.632 \times (-17.4) = -11.0°$$

so 0.4 s later the wheels are at $-11.0°$, not $-17.4°$. At 5 m/s the car drives 2 m in those 0.4 s, half of a 4 m look-ahead and the whole of a 2 m one. Figure 7 sweeps the look-ahead again at 5 m/s with this steering. Watch the red 2 m car: each correction arrives late, overshoots, and is corrected late again.

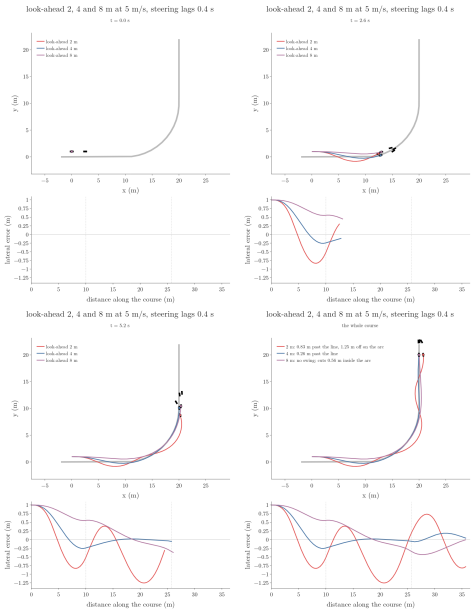


Figure 7: The look-ahead distance swept at 5 m/s, with steering that follows the command with a lag of 0.4 s (added to the chapter's car in this section): 2 m (red), 4 m (blue), 8 m (purple), from 1.0 m left of the course; one frame per 0.2 s. Top: paths of the rear axles. Bottom: lateral errors along the course. The 2 m car weaves: 0.83 m past the line, then 1.25 m off on the arc and still 0.82 m on the last straight. The 4 m car swings 0.26 m past the line. The 8 m car does not swing but cuts 0.56 m inside the arc

Look-ahead	Past the line, first straight	Largest on the arc	After the arc
2 m	0.83 m	1.25 m, outside	0.82 m
4 m	0.26 m	0.24 m, outside	0.18 m
8 m	none	0.56 m, inside	0.43 m, wide

The sweep shows the trade-off the sources describe: a short look-ahead tracks closely until it weaves; a long one is smooth but tracks poorly and cuts corners (Coulter §4.1; Snider §2.2.1; MathWorks pure pursuit documentation). Nav2's tuning advice says the same: "If you see wiggling, increase the distance or scale. If it's not converging as fast to the path as you'd like, decrease it" (Nav2 RPP README).

The same lag hurts much less at 2 m/s: there the 4 m look-ahead crosses the line by only 0.037 m, against 0.26 m at 5 m/s (notebook). So a look-ahead that suits one speed weaves at another.

6. A look-ahead that grows with speed

> **Key point:** Anything that takes time, such as the 0.4 s lag of the steering motor added in Section 5.2, becomes distance at speed: 0.8 m at 2 m/s, 2 m at 5 m/s. So the look-ahead is set to the speed times a time, here 2 s, which gives 4 m at 2 m/s and 10 m at 5 m/s, kept between limits: at 5 m/s the car stops weaving, but the 10 m look-ahead cuts the arc.

We keep the 0.4 s steering lag of Section 5.2, added to the chapter's car because real steering has one. With it, at 2 m/s the 4 m look-ahead distance, how far ahead the chased point lies, crosses the line by 0.037 m; at 5 m/s, by 0.26 m. In 0.4 s the car drives 0.8 m at 2 m/s but 2 m at 5 m/s, so the lag eats into the look-ahead more and more as the car speeds up. Paden et al. (§V.A.1) give the same reason from the turn rate. With:

- v, the car's speed, in m/s;
- ω, the rate at which the car's heading turns, in rad/s: the speed times the curvature of Section 3.2;
- α, the angle from the heading to the look-ahead point, as in Section 3.2.

$$\omega = v\,\kappa$$

$$= \frac{2v \sin\alpha}{l_d}$$

At a fixed look-ahead, the same angle gives a turn rate that grows with the speed, and "a common fix for this issue is to scale L with the vehicle speed" (Paden §V.A.1).

The fix is a look-ahead distance proportional to the speed, kept between a smallest and a largest value: a **velocity-scaled look-ahead** (G-3153), whose pure pursuit Macenski et al. (§2) call adaptive pure pursuit. With:

- v, the car's speed, in m/s, as above;
- t_{la}, the look-ahead time, 2 s, our choice so that the base speed keeps the base 4 m;
- l_d, the look-ahead distance, kept between 2 and 10 m.

$$l_d = v\,t_{la}$$

$$2 \times 2 = 4 \text{ m at } 2 \text{ m/s}$$

$$5 \times 2 = 10 \text{ m at } 5 \text{ m/s}$$

Put into the turn rate, the speed cancels:

$$\omega = \frac{2v \sin \alpha}{v\, t_{la}}$$

$$= \frac{2 \sin \alpha}{t_{la}}$$

So the same angle gives the same turn rate at every speed, and the car turns as briskly in time at 5 m/s as at 2 m/s.

Real trackers use the same idea with their own numbers (Figure 8):

- **Snider (§2.2.1):** the look-ahead is a gain times the speed, kept between 3 and 25 m.
- **Nav2:** `lookahead_time` 1.5 s, kept between 0.3 and 0.9 m, for robots that drive at most 0.5 m/s (parameter defaults in `parameter_handler.cpp`).
- **Autoware:** 2.4 s times the speed, minus 120 m^2 times the path's curvature, plus 3.6 times the lateral error when that is 0.5 m or more, kept between 4.35 and 15 m (`calcLookaheadDistance`, `pure_pursuit.param.yaml`). At 5 m/s that is 12 m on a straight but 4.35 m on our 10 m arc, where the curvature takes 12 m off.

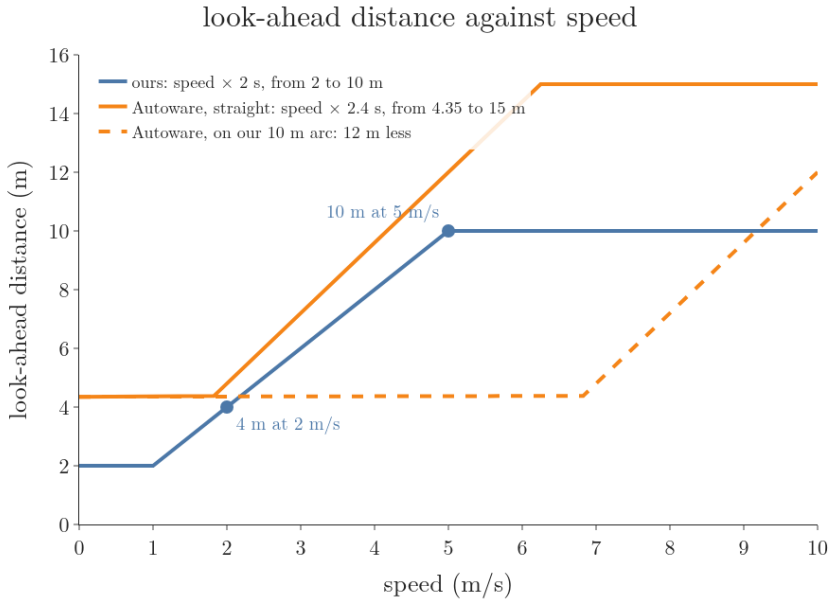


Figure 8: Look-ahead distance against speed. Blue: ours, the speed times 2 s, kept between 2 and 10 m: 4 m at 2 m/s and 10 m at 5 m/s. Orange: Autoware's pure pursuit, 2.4 s times the speed kept between 4.35 and 15 m, on a straight (solid) and on our 10 m arc (dashed), where its curvature term takes 12 m off

At 5 m/s with the steering lag, the 10 m look-ahead no longer swings past the line. But it returns slowly, within 0.1 m only after 22.6 m, and it cuts the arc: 0.98 m inside at the entry, where it is still coming back from the start, 0.60 m in the middle of the arc, and 0.76 m wide after it (notebook). Autoware's curvature term shows the way out: shorten the look-ahead where the path bends. Slowing down there does the same, and does more.

7. Regulated pure pursuit: slow down where the path is tight

> **Key point:** Regulated pure pursuit, the version in Nav2, lowers the speed in proportion to the radius of the arc it commands whenever that arc is tighter than a set radius, here 25 m (so 2 m/s on our 10 m arc), and also near obstacles; with a look-ahead that grows with speed, slowing also shortens the look-ahead. In the middle of the arc the car is 0.27 m off instead of 0.60 m, and 0.14 m instead of 0.76 m after it, for 11.4 s on the course instead of 7.0 s.

At a constant 5 m/s the 10 m look-ahead cuts the arc by 0.60 m in its middle; this section keeps the 0.4 s steering lag added in Section 5.2, so that speed costs accuracy as it does on a real car. At a constant 2 m/s, with the same steering lag, the 4 m look-ahead is 0.00 m off in the middle of the arc and 0.10 m after it, but the course takes 17.9 s. A driver does neither: a driver slows for the bend and speeds up on the straight. Macenski et al. (§3) build that habit into pure pursuit.

Regulated pure pursuit (G-3154) adds two rules to pure pursuit that lower the speed (Macenski et al. 2023, §3):

- **the curvature rule:** when the arc pure pursuit commands is tighter than a set radius, the speed is cut in proportion to its radius;
- **the proximity rule:** when an obstacle is closer than a set distance, the speed is cut in proportion to that distance.

The curvature rule as Nav2 codes it (`curvatureConstraint` in `regulation_functions.hpp`), with:

- $v_{\max}$, the speed asked for, 5 m/s;
- R, the radius of the arc pure pursuit commands, one over the size of its curvature, in m (always positive here, unlike the signed R of Section 3.2);
- $r_{\min}$, the radius below which the rule acts, 25 m: our choice, so that an arc as tight as our 10 m one is driven at the base speed, 2 m/s;
- v, the speed the rule allows, never below 0.25 m/s (Nav2's `regulated_linear_scaling_min_speed`).

When R is below $r_{\min}$:

$$v = v_{\max}\, R / r_{\min}$$

$$5 \times 10/25 = 2 \text{ m/s on a 10 m arc}$$

$$5 \times 5/25 = 1 \text{ m/s on a 5 m arc}$$

Figure 9 draws the rule. The proximity rule has the same shape, with the distance to the nearest obstacle in place of R: with Nav2's defaults (`cost_scaling_dist` 0.6 m, `cost_scaling_gain` 1.0), an obstacle 0.3 m away halves the speed, from 0.5 to 0.25 m/s.

Our Python model codes only the curvature rule, in Nav2's form; our course has no obstacles, so the proximity rule would not act. Our car also changes speed by at most 2.5 m/s^2, Nav2's default `max_linear_accel`.

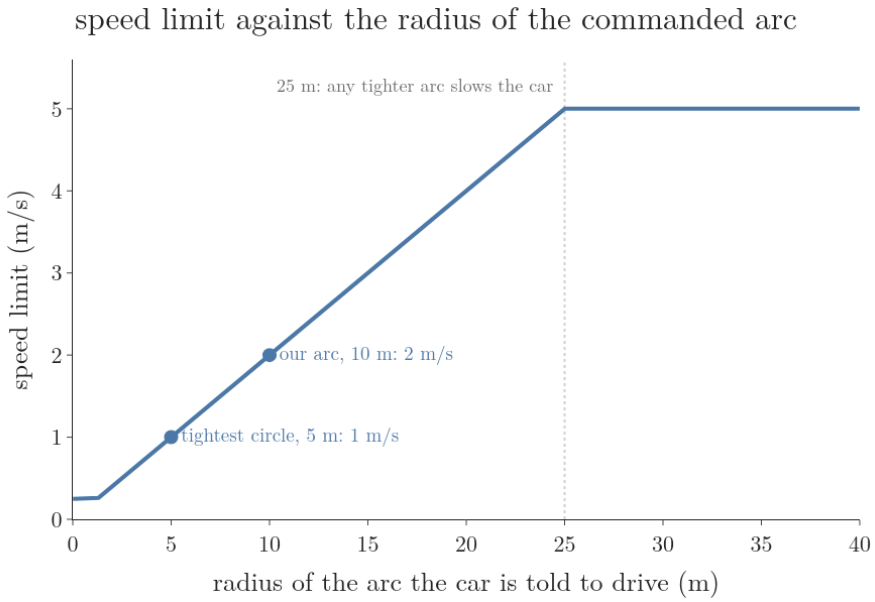


Figure 9: Regulated pure pursuit's curvature rule with a requested speed of 5 m/s and a set radius of 25 m: below 25 m the speed limit falls in proportion to the radius of the commanded arc, to 2 m/s for our 10 m arc and 1 m/s for the car's tightest 5 m circle, but never below 0.25 m/s

Slowing helps twice. A slower car has less to lose to the steering lag, and its velocity-scaled look-ahead, the speed times 2 s, shrinks, so it cuts less. Figure 10 races the two cars at 5 m/s with the steering lag, both from the start. Watch the blue car's dashed look-ahead circle: it shrinks as the car slows, from about 7 m down the course, to 2.8 m a metre past the middle of the arc, where the car is slowest, 1.41 m/s.

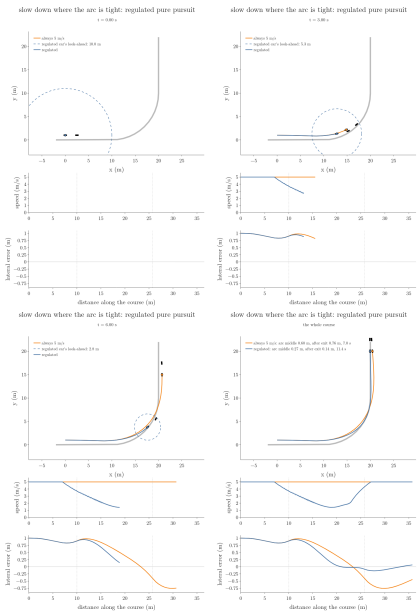


Figure 10: Regulated pure pursuit (blue) against plain pure pursuit at a constant 5 m/s (orange), both with the look-ahead of speed times 2 s and steering that lags 0.4 s (added to the chapter's car in Section 5.2), from 1.0 m left of the course; one frame per 0.25 s. Top: the paths; the dashed circle is the blue car's look-ahead, which shrinks as it slows. Middle: speed along the course: the blue car slows from about 7 m on, down to 1.41 m/s in the arc, and speeds up after it. Bottom: lateral errors. Last frame: middle of the arc 0.60 m (orange) against 0.27 m (blue); after the arc 0.76 m against 0.14 m; 7.0 s against 11.4 s

Driving, all with the 0.4 s steering lag	Middle of the arc	After the arc	Whole course
always 5 m/s, look-ahead speed × 2 s	0.60 m	0.76 m	7.0 s
regulated, from 5 m/s, look-ahead speed × 2 s	0.27 m	0.14 m	11.4 s
always 2 m/s, look-ahead 4 m	0.00 m	0.10 m	17.9 s

The regulated car cuts the fast car's error by more than half in the middle of the arc and by four fifths after it, and needs about two thirds of the slow car's time. The regulated car slows below 2 m/s, to 1.41 m/s, because while it turns back from cutting the arc's entry, the arc it commands is tighter than the course's 10 m.

The rule does not help at the arc's entry: both cars reach it still about 0.9 m inside, returning from the start with a 10 m look-ahead, and run up to 0.98 m (orange) and 0.95 m (blue) inside just after it (notebook). The curvature rule reacts to the arc pure pursuit commands, and that

arc bends only once the look-ahead point is round the corner. Nav2 offers a separate, fixed look-ahead just for reading the curvature (`use_fixed_curvature_lookahead`) for robots with long look-aheads.

8. Where pure pursuit runs: Nav2 and Autoware

> **Key point:** Nav2's Regulated Pure Pursuit controller and Autoware's pure pursuit lateral controller both find a look-ahead point, the path point a set distance ahead, and command the curvature of the arc through it; both can make the look-ahead grow with speed, and Nav2 adds the slowing rules of Section 7.

Two open-source robot stacks run the rules of Sections 2 to 7, which our runs only modelled in Python. Each fact below comes from the stack's code or parameter files; their paths are in Sources.

Nav2, the ROS 2 navigation stack: `nav2_regulated_pure_pursuit_controller`. The controller is one of Nav2's plugins, the local planner (G-2911) layer that turns the global plan into velocity commands (Nav2's default configuration uses its MPPI controller instead). Each call:

1. prunes the path to the point closest to the robot and turns the nearby part into the robot's frame (Nav2 RPP README);
2. finds the look-ahead point: `lookahead_dist` 0.6 m, or, with `use_velocity_scaled_lookahead_dist`, the speed times `lookahead_time` 1.5 s kept between 0.3 and 0.9 m (`getLookAheadDistance`);
3. computes the curvature of Section 3.2, twice the point's sideways distance over its squared distance from the robot (`calculateCurvature`);
4. applies the curvature rule below `regulated_linear_scaling_min_radius` 0.9 m and the proximity rule within `cost_scaling_dist` 0.6 m, never below 0.25 m/s, and at most `max_linear_vel` 0.5 m/s (`applyConstraints`);
5. sends a turn rate equal to the speed times the curvature, since most Nav2 robots are differential drives, which drive an arc by their linear and angular velocity;
6. checks the arc it commands for collisions up to 1.0 s ahead (`max_allowed_time_to_collision_up_to_`

Autoware, the open-source self-driving stack: `autoware_pure_pursuit`. The package is one of the two lateral controllers of `autoware_trajectory_follower_node`; the node's `lateral_controller_mode` picks MPC or pure pursuit. The package takes the planned trajectory and the car's odometry and returns a steering angle (Autoware pure pursuit README). Its look-ahead is the rule of Section 6: it grows with speed (2.4 s), shrinks on curves (120 m^2 times the curvature), grows when the car is far off the path (3.6 times a lateral error of 0.5 m or more) and stays between 4.35 and 15 m, or at least 7.0 m when reversing (`pure_pursuit.param.yaml`).

9. Summary

Idea	What it is	Our numbers
Look-ahead point	the path point one look-ahead distance from the rear axle, ahead of the nearest point	(3.873, 0) at the start, with 4 m
The arc	leaves along the heading, so its centre is on the rear-axle line; $\kappa = 2y/l_d^2 = 2\sin\alpha/l_d$	-0.125 per m, a circle of 8 m to the right
Steering	$\delta = \tan^{-1}(L\kappa)$	$-17.4°$ at the start
Why no weaving	the point's sideways distance holds the lateral error and the heading error: a spring and a damper	damping ratio 0.707 for every look-ahead
Look-ahead trade-off	short weaves when the steering lags, long cuts corners	at 5 m/s with the added 0.4 s lag: 2 m weaves 1.25 m off, 8 m cuts 0.56 m
Velocity-scaled look-ahead	$l_d = v\,t_{la}$, kept between limits	4 m at 2 m/s, 10 m at 5 m/s
Regulated pure pursuit	speed cut to $v_{\max}R/r_{\min}$ below $r_{\min}$, and near obstacles	2 m/s on the 10 m arc; 0.27 m off mid-arc instead of 0.60 m

- The opening question has its answer: on the 10 m arc pure pursuit is 0.03 m off by the middle, where the line rule stays 0.36 m outside (up to 0.39 m), because once the look-ahead point is on the arc the arc through it is the course's own arc, so the car turns with the bend without needing an error; from 1.0 m off it is back within 0.1 m after 7.4 m, against 7.70 m for the line rule.
- The look-ahead point is where a circle of radius l_d around the rear axle meets the path ahead, so the rule needs only the path's points and the car's pose.
- The car can only drive arcs, and the drivable arc leaves along the heading, so its centre lies on the rear-axle line and one right triangle gives its curvature, twice the point's sideways distance over the look-ahead distance squared, which the bicycle model turns into the steering angle.
- Near the path, pure pursuit is a spring on the lateral error and a damper on the heading error, with the damping ratio 0.707 whatever the look-ahead, so on a car that steers at once it swings past the line by at most 4.3 percent of its start error, and a shorter look-ahead only makes it settle sooner (4.0 m for 2 m, 7.4 m for 4 m).
- On a long arc any look-ahead gives the arc's curvature, so the errors come where the curvature changes: a long look-ahead turns early and cuts corners (0.85 m inside for 8 m).
- A steering motor that lags 0.4 s, which we added to the chapter's car because real steering lags, makes a short look-ahead weave, more at speed, because the lag becomes 2 m of travel at 5 m/s, so the look-ahead must grow with speed: the speed times 2 s

here, between 3 and 25 m in Snider's car, 4.35 to 15 m in Autoware.
- A long look-ahead at speed still cuts corners, so regulated pure pursuit slows down on arcs tighter than a set radius (25 m here) and near obstacles; slowing also shortens the velocity-scaled look-ahead, which cut the error in the middle of the arc from 0.60 to 0.27 m for 4.4 s more on the course.

The car can now follow its path from a small lateral error and choose its look-ahead and speed for the turns ahead. Still missing: pure pursuit never looks at the path's direction or curvature at the car itself, so it cuts corners and its look-ahead must be retuned for every speed. Steering from the errors at the car's own wheels is the Stanley controller and rear-wheel feedback.

10. Sources

Built from

- Neuromorphic Workshop Telluride, "LTC21 Tutorial Pure Pursuit", YouTube, https://www.youtube.com/watch?v=qYR7mmcwT2w
- MATLAB, "Vehicle Path Tracking Using Pure Pursuit Controller", YouTube, https://www.youtube.com/watch?v=zMdoLO4kRKg
- Coulter, R. C. (1992). *Implementation of the Pure Pursuit Path Tracking Algorithm.* Technical report CMU-RI-TR-92-01, Robotics Institute, Carnegie Mellon University. §1.2 Description, §2.0 Theoretical derivation, §3.3 Pursuit algorithm, §4.1 Effects of changing the lookahead distance, §4.2 Non-unique lookahead for a given path curvature, §4.3 Comments. https://www.ri.cmu.edu/pub_files/pub3/coulter_r_craig_1992_1/coulter_r_craig_ (Coulter)
- Snider, J. M. (2009). *Automatic Steering Methods for Autonomous Automobile Path Tracking.* Technical report CMU-RI-TR-09-08, Robotics Institute, Carnegie Mellon University. §2.1 Geometric vehicle model, §2.2 Pure pursuit, §2.2.1 Tuning the pure pursuit controller. https://www.ri.cmu.edu/pub_files/2009/2/Automatic_Steering_Methods_for_Autonomo (Snider)
- Paden, B., Čáp, M., Yong, S. Z., Yershov, D. and Frazzoli, E. (2016). "A Survey of Motion Planning and Control Techniques for Self-driving Urban Vehicles". *IEEE Transactions on Intelligent Vehicles* 1(1). §V.A.1 Pure pursuit. https://arxiv.org/abs/1604.07446 (Paden)
- Macenski, S., Singh, S., Martín, F. and Ginés, J. (2023). "Regulated Pure Pursuit for Robot Path Tracking". *Autonomous Robots* 47. §2 (adaptive pure pursuit, eq. 4), §3 (the curvature and proximity heuristics, eqs. 5–7). https://arxiv.org/abs/2305.20026 (Macenski et al.)
- Hoffmann, G. M., Tomlin, C. J., Montemerlo, M. and Thrun, S. (2007). "Autonomous Automobile Trajectory Tracking for Off-Road Driving: Controller Design, Experimental Validation and Racing". *American Control Conference.* §II.B, eq. 4 (the steering as a first-order lag, $\tau = 0.4$ s in their simulations). http://ai.stanford.edu/~gabeh/papers/hoffmann_stanley_control07.pdf (Hoffmann et al. 2007)
- Gupta, A. and Srinivasa, S., University of Washington CSE 478 *Autonomous Robotics*,

Lecture 11 “Pure Pursuit” slides, https://courses.cs.washington.edu/courses/cse478/26sp/lectures/Lect (UW CSE 478 L11)

- Code and parameter files read for Sections 6 to 8:
  - Nav2: nav2_regulated_pure_pursuit_controller/src/regulated_pure_pursuit_controller.cpp in ros-navigation/navigation2
  - Nav2: nav2_regulated_pure_pursuit_controller/include/nav2_regulated_pure_pursuit_contro in ros-navigation/navigation2
  - Nav2: nav2_regulated_pure_pursuit_controller/src/parameter_handler.cpp in ros-navigation/navigation2
  - Nav2: nav2_regulated_pure_pursuit_controller/README.md in ros-navigation/navigation2 (Nav2 RPP README)
  - Nav2: nav2_bringup/params/nav2_params.yaml in ros-navigation/navigation2
  - Autoware: control/autoware_pure_pursuit/src/autoware_pure_pursuit/autoware_pure_pursuit in autowarefoundation/autoware_universe
  - Autoware: control/autoware_pure_pursuit/config/pure_pursuit.param.yaml in autowarefoundation/autoware_universe
  - Autoware: control/autoware_pure_pursuit/README.md in autowarefoundation/autoware_universe (Autoware pure pursuit README)

Other references

- MathWorks, “Pure Pursuit Controller”, Navigation Toolbox documentation (the look-ahead distance as the main tuning property), https://www.mathworks.com/help/nav/ug/pure-pursuit-controller.html (MathWorks pure pursuit documentation)
- Sakai, A. et al., PythonRobotics, “Pure pursuit tracking” (look-ahead 0.1 s times the speed plus 2.0 m), https://github.com/AtsushiSakai/PythonRobotics/blob/master/PathTracking/pure
- Corke, P., *Robotics, Vision and Control* (3rd ed., Python), model `drivepursuit.py` in RVC3-python, https://github.com/petercorke/RVC3-python/blob/main/RVC3/models/drivepursuit.p

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Look-ahead distance (pure pursuit) (G-3151)	The distance l_d from the rear axle to the point pure pursuit chases, such as 4 m; a short one brings the car back sooner but weaves when the steering lags, a long one is smooth but cuts corners.

Term	Meaning
Look-ahead point (pure pursuit) (G-3152)	The point of the path one look-ahead distance from the rear axle, found by walking forward from the nearest path point, such as (3.873, 0) for a car 1 m off a straight line with a 4 m look-ahead; pure pursuit steers along the arc that reaches it. Also called the goal point or carrot.
Pure pursuit (G-3150)	A path tracker that, at every tick, finds the point on the path one look-ahead distance from the rear axle and steers along the circle that leaves the car along its heading and passes through that point, of curvature $2y/l_d^2$; looking ahead brings the heading error into the steering, so a car whose steering answers at once returns to the path with little swing past it.
Velocity-scaled look-ahead (adaptive pure pursuit) (G-3153)	A look-ahead distance set to the speed times a look-ahead time and kept between limits, such as 2 s giving 4 m at 2 m/s and 10 m at 5 m/s; it keeps pure pursuit from weaving at speed, where the same steering delay covers more distance.
Regulated pure pursuit (G-3154)	Pure pursuit with rules that lower the speed when the commanded arc is tighter than a set radius, to $v_{\max}R/r_{\min}$, and when an obstacle is near, as in Nav2; with a velocity-scaled look-ahead, slowing also shortens the look-ahead, so the car cuts tight turns less.
Path coordinates (Frenet frame) (G-3132)	Giving a point's place by the distance along a path to its nearest path point and the signed distance to the side of it, measured along the path's tangent and normal there, the Frenet frame that slides along the path; they let a tracker treat any bending path like a straight line.
Nearest path point (G-3134)	The point of a path closest to a chosen point of the vehicle, such as its rear axle; trackers measure the lateral and heading errors there, and on a path that comes back near itself the plain nearest point can jump, so real stacks limit the search.

Term	Meaning
Lateral error e (cross-track error) (G-3135)	The signed distance from a path to a point of the vehicle, measured along the path's normal at the nearest path point and positive to the left, such as -0.770 m for a car outside an arc; it is one of the two numbers a path tracker drives to zero.
Heading error θ_e (G-3136)	A vehicle's heading minus the path's direction at the nearest path point, wrapped into $-180°$ to $180°$, such as $10° - 21.8° = -11.8°$; it says whether the vehicle is turning back to the path or away, and the lateral error changes at $v \sin \theta_e$.
Wheelbase L (G-3127)	The distance from a car's rear axle to its front axle, such as 2.5 m; with the steering angle it fixes the turning radius, $R = L/\tan \delta$, so a longer car turns more widely for the same steering.
Kinematic bicycle model (G-3128)	A car drawn as one steered front wheel and one rear wheel on its centre line, with no tyre slip: the rear axle moves along the heading and the heading turns at $\dot{\theta} = v \tan \delta / L$; it predicts a car's motion well while the sideways acceleration stays below about half the road's friction (grip) coefficient times gravity.
Sample period and sample rate (G-3106)	The time T between two samples, and its inverse, the number of samples per second $f_s = 1/T$ in hertz, such as 0.01 s and 100 Hz; they set how often a digital controller sees the robot and acts.
Instantaneous centre of curvature (ICC) (G-2529)	The point a turning robot's whole body circles at a given moment; for a differential drive it lies on the axle line at distance $R = v/\omega$ from the midpoint, so it fixes how fast each wheel must move.
Path curvature κ (G-3129)	How fast a path's direction turns per metre along it, $1/R$ on an arc of radius R, such as 0.1 per m on a 10 m arc and 0 on a straight; it fixes the steering a bend needs, $\tan \delta = L\kappa$.
Steering angle δ (G-3126)	The angle between a car's steered front wheel and its heading, positive to the left, such as at most $26.6°$ for RO-055's car; it sets how sharply the car turns, through $\tan \delta = L/R$.

Term	Meaning
Zero-order hold (G-3109)	Keeping each command constant from its tick until the next one, so the actuator gets a staircase; simple and cheap, but the held value is on average half a sample period old, so it acts like a delay of $T/2$.
Derivative action (G-2960)	A part of the command in proportion to the error's rate of change, estimated from two readings in a row; it pushes against fast changes, so it brakes a turn before the target and damps swings, but it magnifies sensor noise.
Natural frequency (G-2991)	The rate, in rad/s, at which a second-order system would swing with no damping, $\omega_n = \sqrt{K_p/I}$, 2 rad/s for the heading loop; it sets how fast the reply is, and raising it squeezes the whole reply in time.
Damping ratio (G-2992)	A second-order system's damping compared with the damping that just stops it from swinging, $\zeta = K_d/(2\sqrt{K_p I})$, 0.2 for D gain 0.08 and 1 for 0.4; it alone sets the shape of the reply and its overshoot.
Critically damped (G-2995)	A second-order system with damping ratio exactly 1: its characteristic equation has one double real root, so it gives the fastest step response that does not overshoot, such as the turn with K_p = K_d = 0.4 settling in 2.92 s.
First-order system (G-2987)	A system whose output follows a differential equation with only a first derivative, such as the robot's speed under P control; it closes the same fraction of its remaining gap in every time constant and can never overshoot.
Time constant (G-2988)	The time a first-order system takes to close 63 % of the gap to its final value, m/(K_v + b) = 1 s for the robot's speed loop; it fixes the rise time (about 2.2 time constants) and the 2 % settling time (about 4).

Term	Meaning
Local planner (G-2911)	The planner that turns the next few metres or seconds of the global plan into a timed trajectory within the robot's limits and around obstacles its sensors see, solving again many times a second, such as TEB at 10 Hz over 1.5 m; it handles what the global plan cannot.